Fundamental Programming Structures in JAVA

COMPOSITION & AGGREGATION

Objectives

- To discover the relationships between classes.
- To distinguish between inheritance and composition.
- To distinguish between aggregation and composition.
- To declare a class has an property of another class.

Introduction

- Relationships between classes are crucial in object-oriented programming. Just as the concepts like classes and objects in object-oriented programming are built to model real-world entities, the relationships between classes in object-oriented programming are built to model the relationships between real-world entities, that these classes represent.
- In a way, almost everything that we see in the real world can be viewed as a bunch of objects and the relationships/associations between them.

Introduction

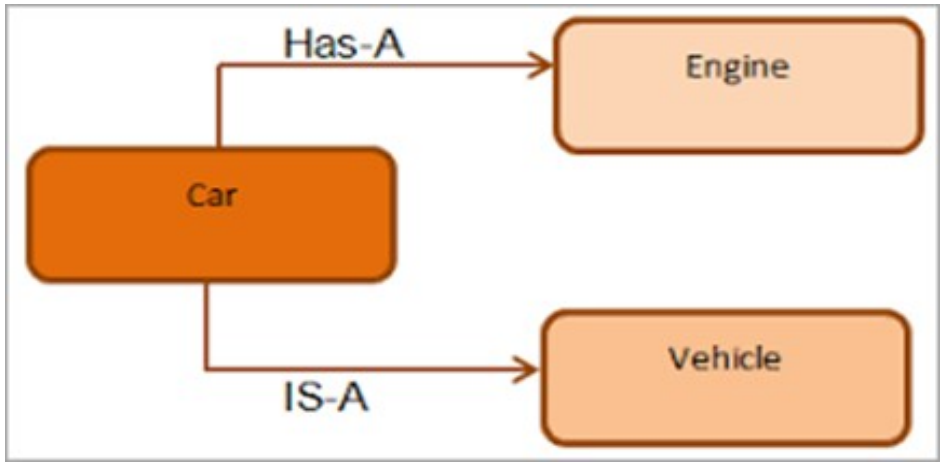
- Classes as individual entities(without any relations between them) are impractical in object-oriented programming because then we would not be able to model the relationships that exist between real-life entities.
- A student is called a student because he studies in a school/college. Here both student and college entities can be modeled as two individual classes with a relation of client-provider (wherein college “provides” education to the student) between them. Similarly, there would be no concept of the employee in real life without the existence of an organization that employs individuals, thus an organization is associated with an employee(here the relation is ‘employment’).

Association in java

- Association, in general terms, refers to the relationship between any two entities. Association in java is the relationship that can be established between any two classes
- Association can be one-to-one, one-to-many, many-to-one, many-to-many. In Object-Oriented programming, an Object communicates to another object to use functionality and services provided by that object.
  - It refers to how objects are related to each other and how they are using each other's functionality.

Forms of Association in Java

- Before we dive into the forms of Association in java, let us briefly explore the types of object relationships that can exist in OOPs. There can be two types of relationships in OOPs:
  - **IS-A**
  - **HAS-A**



Forms of Association in Java

- The **IS-A** relationship is nothing but Inheritance. The relationships that can be established between classes using the concept of inheritance are called IS-A relations.
- Ex: A parrot is-a Bird. Here Bird is a **base class**, and Parrot is a **derived class**, Parrot class inherits all the properties and attributes & methods (other than those that are private) of base class Bird, thus establishing inheritance(IS-A) relation between the two classes. ***It will be explained in the eighth week material.***

Forms of Association in Java

- The **HAS-A** association on the other hand is where the Instance variables of a class refer to objects of another class. In other words, one class stores the objects of another class as its instance variables thereby establishing a HAS-A association between the two classes.

Forms of Association in Java

- There are two forms of Association that are possible in Java:
  - **Aggregation**
  - **Composition**

Forms of Association in Java-Aggregation

- Aggregation in java is a form of HAS-A relationship between two classes. It is a relatively more loosely coupled relation than composition in that, although both classes are associated with each other, one can exist without the other independently. So Aggregation in java is also called **a weak association**.
- Aggregation represents HAS-A relationship.
  - Consider a situation, Employee object contains many information such as id, name, email ,Id etc.
  - It contains one more object named address, which contains its own information's such as city, state, country, zipcode etc. as given below

Aggregation Example

- In this example, Employee has an object of Address, address object contains its own information such as city, state, country etc.
  - In such case relationship is Employee **HAS -A** address

Aggregation Example

```
public class Address {                    //Address.java
String city,state,country;

public Address(String city, String state, String country) {
    this.city = city;
    this.state = state;
    this.country = country;
}

}
```

```
public class Emp {                                        //Emp.java
int id;
String name;
Address address;

public Emp(int id, String name,Address address) {
    this.id = id;
    this.name = name;
    this.address=address;
}

void display(){
System.out.println(id+" "+name);
System.out.println(address.city+" "+address.state+" "+address.country);
}
}
```

Aggregation

Aggregation Example

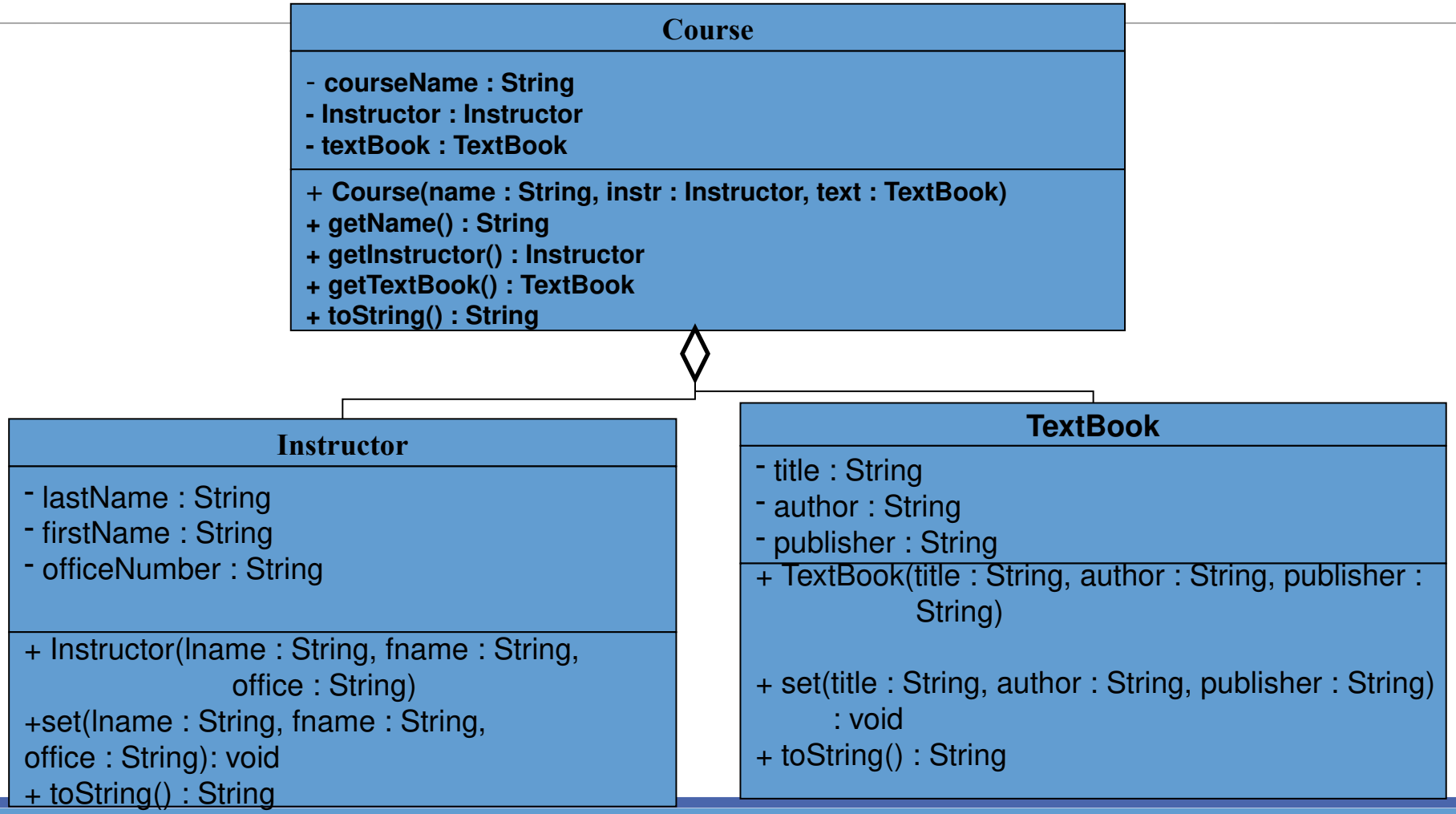
```
//AggregationTest.java
public class AggregationTest { publicstaticvoidmain(String[]args) {

Addressaddress1=newAddress(“khalda",“Amman",“Jordan");
Addressaddress2=newAddress(“abdoon",“Amman",“Jordan");

Empe=newEmp(111,“Abdullah",address1);
Empe2=newEmp(112,“Ali",address2);

e.display();
e2.display(); } }
```

Aggregation (Course, Instructor & TextBook)



Aggregation Example

Instructor

- lName : String
- fName : String
- officeNumber : String

+ Instructor(lname : String, fname : String, office : String)

+set(lname : String, fname : String, office : String): void
+ toString() : String

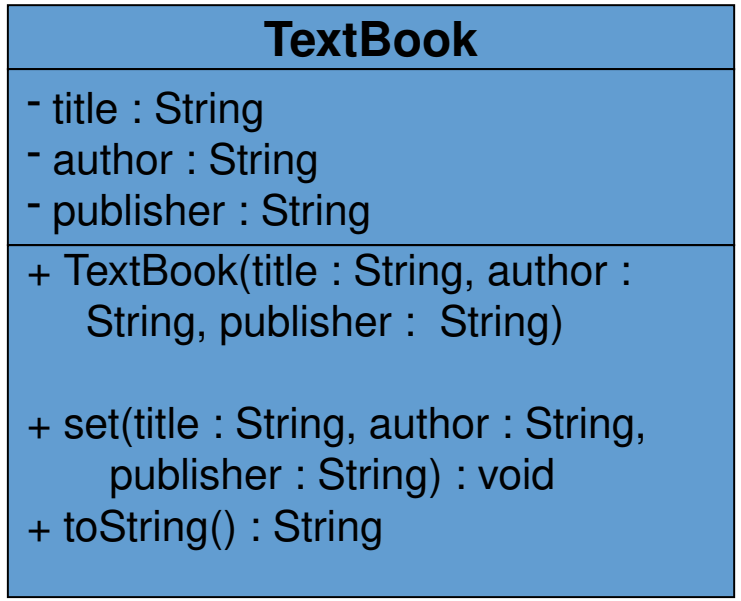
```
public class Instructor {
          private String lName;
          private String fName;
          private String officeNum;

  public Instructor(String lName, String fName, String officeNum)
          {
          this.lName = lName;
          this.fName = fName;
          this.officeNum = officeNum;
          }

      public String getLName()
          {   return lName;   }
      public String getFName()
          {    return fName;  }
      public String getONum()
          {   return officeNum;   }

      public String toString()
          {  String str = "Last Name: " + lName + "\nFirst Name: "
            + fName + "\nOffice Number: " + officeNum + "\n";
          return str;     }
}
```

Aggregation Exam




```
public class TextBook
{
    private String title;
    private String author;
    private String publisher;
    public TextBook(String title, String author, String
publisher)
    {
    this.title = title;
    this.author = author;
    this.publisher = publisher;
    }
    public String getTitle()
    {     return title;   }
    public String getAuthor()
    {     return author;   }
    public String getPublisher()
    {     return publisher;  }
    public String toString()
    {
    String str = "Title: " + title + "\nAuthor: "
             + author + "\nPublisher: " + publisher
+"\n";
    return str;
    }}
```

Aggregation Exam

Course
- courseName : String - Instructor : Instructor - textBook : TextBook
+ Course(name : String, instr : Instructor, text: TextBook) + getName() : String + getInstructor() : Instructor + getTextBook() : TextBook + toString() : String

```
public class Course
{
      private String courseName;
      private Instructor instructor;
      private TextBook textbook;

      public Course(String name, Instructor instr, TextBook
text){
     courseName = name;
     instructor = instr;
     textbook = text;}
      public String toString()
     {
      String str = "Course name:
"+courseName+"\n\nInstructor Information:\n"
            +instructor+"\nTextbook Information:
\n"+textbook;
     return str;
         }
      public void showBookInfo() {
        System.out.println("The book title is " +
textbook.title +
                        "\n The book author is " +
textbook.author +
                  "\n The book publisher is " +
textbook.publisher);
     }
}
```

Reference variables

Aggregation

Forms of Association in Java-Composition

- Composition in java is a form of relation that is more tightly coupled. Composition in java is also called Strong association.
- This association is also known as Belongs-To association as one class, for all intents and purpose belongs to another class, and exists because of it.
- In a Composition association, the classes cannot exist independent of each other. If the larger class which holds the objects of the smaller class is removed, it also means logically the smaller class cannot exist. Let us explore this association clearly with an example

Composition Example

- **Example**: The association between College and Student. Below is how it is defined.
  - College class is defined with name and the list of students that are studying in it
  - A Student class is defined with name and the college he is studying at.
- Here a student must be studying in at least one college if he is to be called Student. If the college class is removed, Student class cannot exist alone logically, because if a person is not studying in any college then he is not a student.

Composition Example

- **Example** a library can have no of books on same or different subjects.
  - So, If Library gets destroyed then All books within that particular library will be destroyed. i.e. book can not exist without library.
  - That's why it is **composition**

Composition Example

```java
public class TeamsAccount {
    private String Email;
    private String password;

    public TeamsAccount(String Email,String password)
    {

   // this.Email=Email;
        this.setEmail(Email);
   // this.password=password;
        this.setPassword(password);

    }
    public String getEmail(){return Email;}
    public String getPassword(){return password;}
```

```java
public void setEmail(String Email)
{
    if(Email.contains("@"))
     this.Email=Email;
    else
        System.out.println("invaliud Email");

}
public void setPassword(String password)
{
    if(password.length()==8)
        this.password=password;
    else
        System.out.println("invalid password");
}
public String toString(){

return "Email : "+this.Email+"\n"+"password : "+this.password;
}
```

Composition Example

```
public class Student {
    private String name;
    private int id;
    private TeamsAccount Team;

public Student(String name,int id,String Email,String password)
    {
        this.name=name;
        this.id=id;
        this.Team=new TeamsAccount(Email,password);

    }
    public String getName() {
        return name;
    }

    public int getId() {
        return id;
    }
```

Reference variable

Composition

Composition Example

```
public void setName(String name) {
    this.name = name;
}

public void setId(int id) {
    this.id = id;
}

public TeamsAccount getTeam(){return Team;}
public void setTeam(String Email,String password){
    this.Team.setEmail(Email);
    this.Team.setPassword(password);

}

public String toString()
{
  return "name : "+this.name+"\nid : "+this.id+"\nAccount : \n "+this.Team;

}
```

Composition

Composition vs Aggregation

- **Dependency**:
  - Aggregation implies a relationship where the child can exist independently of the parent.
    - Example, Bank and Employee, delete the Bank and the Employee still exist.

- whereas Composition implies a relationship where the child cannot exist independent of the parent.
  - Example: Human and heart, heart don’t exist separate to a Human

Composition vs Aggregation

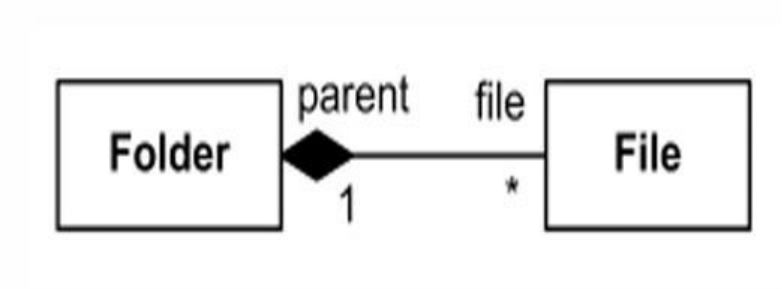
- Type of Relationship:

Aggregation relation is "**has-a**" and composition is "**part-of**" relation.

- Type of association:
  - Composition is a strong Association
  - Aggregation is a weak Association

UML Notation of Composition

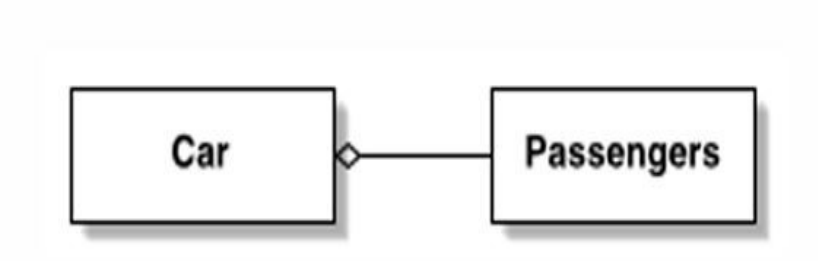
- Composite aggregation is described as a binary association decorated with a filled black diamond at the aggregate (whole) end.



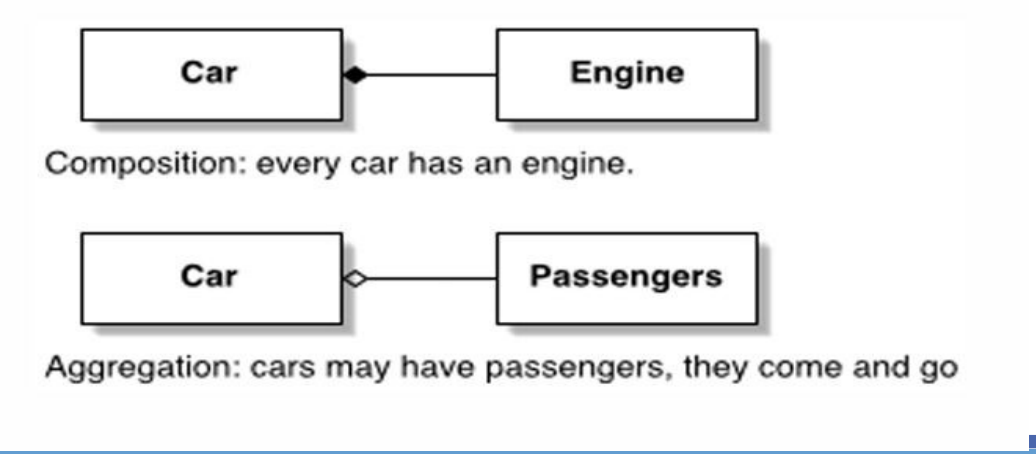
- The folder could contain many files, while each File has exactly one Folder parent.
- If a folder is deleted, all contained files are removed as well

UML Notation of Aggregation

- Aggregation is described as a binary association decorated with an un-filled diamond.



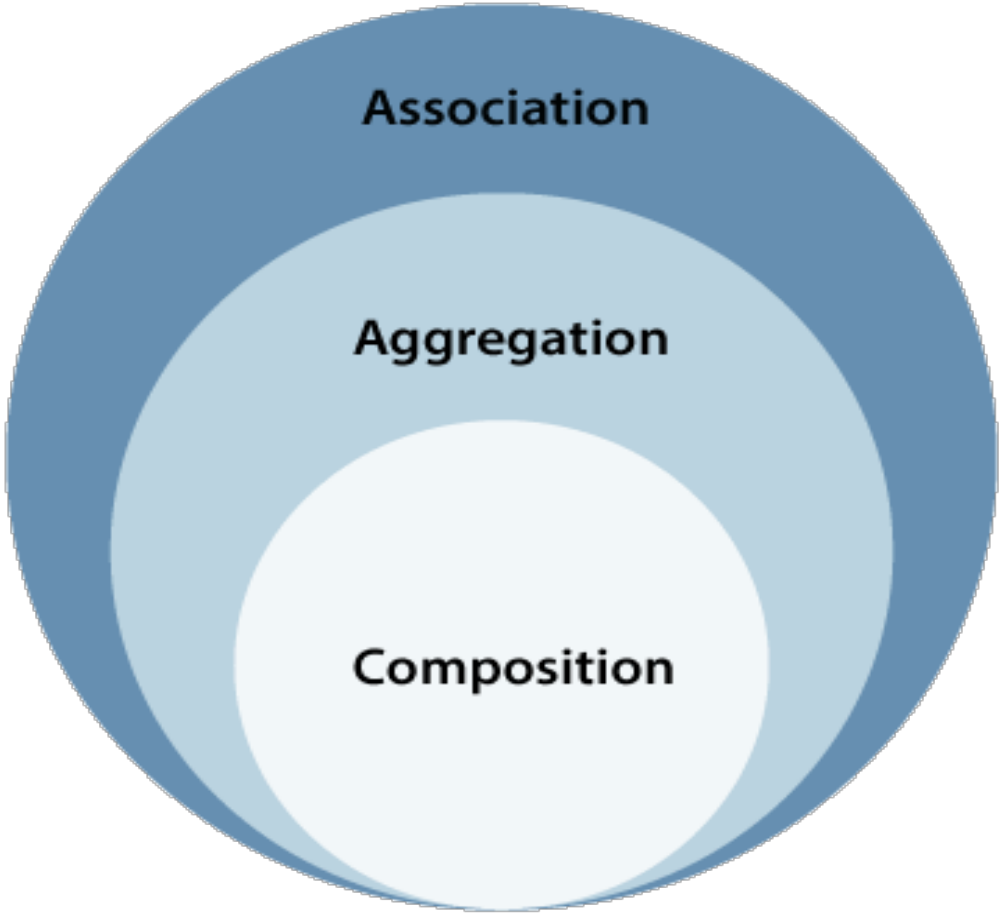
- Car may have passengers; they come and go



Composition vs Aggregation

Aggregation	Composition
1.Weak Association	1.Strong Association
2.Classes in relation can exist independently	2.One class is dependent on Another Independent class. The Dependent class cannot exist independently in the event of the non-existence of an independent class.
3.One class has-a relationship with another class	3.Once class belongs-to another class
4.Helps with code reusability. Since classes exist independently, associations can be reassigned or new associations created without any modifications to the existing class.	4.Code is not that reusable as the association is dependent. Such Associations once established will create a dependency, and these associations cannot be reassigned or new associations like aggregation, etc cannot be created without changing the existing class.

Composition vs Aggregation



References

- [1] Introduction to Java Programming, Comprehensive Version, Student Value Edition, Daniel Liang, 12th Edition, Pearson 2019
- lecture Notes :edugate.asu.edu.jo.
- lecture Videos : YouTube channel.
- Lab manual